

Transfer Protocol

A generalized ownership-transfer protocol for JavaScript

Stage 0



The Problem

Ownership transfer exists in the platform – but there's no common protocol

- `ArrayBuffer.prototype.transfer()` – moves buffer data, detaches the original (ES2024)
- `DisposableStack.prototype.move()` – moves resources to a new stack, marks original disposed
- `ReadableStream` / `WritableStream` locking – `getReader()` locks a stream to a single consumer
- `postMessage()` transfer lists – `ArrayBuffer`, `MessagePort`, `OffscreenCanvas`, etc.
- Runtime-specific types – Node.js adds `FileHandle` as a transferable type (detached on send)

Same problem -- exclusive ownership of a stateful resource -- but no shared protocol. No way to write generic transfer code. No way for user-defined types to participate.

Motivating Example: Iterators

Iterators are stateful and single-consumer, but nothing enforces it

```
const iter = [1, 2, 3, 4].values();

// Two consumers unknowingly share the same iterator
consumeEvens(iter);
consumeOdds(iter);
// Each consumer sees an unpredictable subset of values
```

Same for async iterators:

```
async function processStream(stream) {
  const iter = stream[Symbol.asyncIterator]();
  pipeline1(iter); // starts consuming
  pipeline2(iter); // races with pipeline1 -- data split unpredictably
}
```

Nothing in the language prevents this. Transfer makes ownership explicit.

The Proposal

Four pieces that generalize ownership transfer

Piece 1: `Symbol.transfer`

A new well-known symbol for opting into transfer

An object implements the protocol by providing a `[Symbol.transfer]()` method that:

1. Returns a **new object** that takes ownership of the receiver's state
2. **Detaches** the receiver permanently

```
class OwnedResource {
  #data;
  #detached = false;

  [Symbol.transfer]() {
    if (this.#detached) throw new TypeError('Cannot transfer a detached resource');
    const data = this.#data;
    this.#data = undefined;
    this.#detached = true;
    return new OwnedResource(data);
  }

  [Symbol.dispose]() {
    if (this.#detached) return; // no-op -- nothing to clean up
  }
}
```

Piece 2: `Object.transfer(obj)`

Triggers `[[Transfer]]` on the argument

```
const a = new OwnedResource({ value: 42 });
const b = Object.transfer(a);

a.use(); // throws TypeError: Resource has been detached
b.use(); // { value: 42 }
```

Why not call `[Symbol.transfer]()` directly?

`Object.transfer()` routes through `[[Transfer]]`, which provides:

- Argument type checking (must be an object)
- Protocol validation (`[Symbol.transfer]` must exist, be callable, return object)
- Proxy `transfer` trap dispatch
- Frozen/sealed object rejection

Direct `obj[Symbol.transfer]()` bypasses all of this – available for perf-sensitive paths where the caller has already validated.

Piece 3: `Reflect.transfer(target)`

The low-level MOP counterpart for Proxy trap forwarding

Operation	<code>Object</code> method	<code>Reflect</code> method	Proxy trap
Get	–	<code>Reflect.get()</code>	<code>get</code>
Set	–	<code>Reflect.set()</code>	<code>set</code>
Delete	–	<code>Reflect.deleteProperty()</code>	<code>deleteProperty</code>
Transfer	<code>Object.transfer()</code>	<code>Reflect.transfer()</code>	<code>transfer</code>

```
const proxy = new Proxy(target, {  
  transfer(target) {  
    console.log('transfer intercepted');  
    return Reflect.transfer(target); // forward to default behavior  
  }  
});
```

Piece 4: Proxy `transfer` Trap

Intercepts `[[Transfer]]()` on Proxy exotic objects

Logging:

```
new Proxy(obj, {  
  transfer(target) {  
    console.log('Transferring...');  
    return Reflect.transfer(target);  
  }  
});
```

Access control:

```
new Proxy(obj, {  
  transfer(target) {  
    if (!allowed())  
      throw new TypeError('Not permitted');  
    return Reflect.transfer(target);  
  }  
});
```

Membranes – security boundaries that must intercept *all* MOP operations:

```
transfer(target) {  
  return new Proxy(Reflect.transfer(target), handler); // keep inside the membrane  
}
```


The `[[Transfer]]` Internal Method

New essential internal method – the engine-level mechanism

For ordinary objects:

1. If `IsExtensible(0)` is `false` – throw `TypeError` (cannot transfer frozen/sealed)
2. Let `transferFn` = `GetMethod(0, @@transfer)` – throws if present but not callable
3. If `transferFn` is `undefined` – throw `TypeError` (not transferable)
4. Let `result` = `Call(transferFn, 0)`
5. If `result` is not an `Object` – throw `TypeError`
6. Return `result`

For Proxy exotic objects, `[[Transfer]]()` is intercepted by the `transfer` handler trap instead.

Built-in Transferables: Iterators

All built-in iterator types are transferable out of the box

- Array iterators – `.values()`, `.keys()`, `.entries()`
- Map / Set iterators – `.values()`, `.keys()`, `.entries()`
- String iterators – `"str"[Symbol.iterator]()`
- RegExp String iterators – `.matchAll()`
- Generator and async generator instances
- Iterator Helpers – `.map()`, `.filter()`, `.take()`, etc.
- `Iterator.from()` wrappers

Sync iterators use `Iterator.prototype[@@transfer]`; async iterators use `%AsyncIteratorPrototype[@@transfer]`.

Built-in Transferables: Other Types

Non-iterator types that implement `[Symbol.transfer]()`

Object type	Mechanism
<code>ArrayBuffer</code>	Delegates to <code>ArrayBufferCopyAndDetach</code>
<code>DisposableStack</code> / <code>AsyncDisposableStack</code>	Delegates to <code>.move()</code>

Not transferable (no `[Symbol.transfer]`):

- Plain `{ next() {} }` iterator objects – must implement `[Symbol.transfer]()` explicitly
- Arrays, Maps, Sets – these are *iterables*, not iterators

Iterator Transfer in Action

After transfer, the original is detached

```
const iter = [1, 2, 3].values();
const owned = Object.transfer(iter);

iter.next();    // throws TypeError: Iterator has been detached
owned.next();   // { value: 1, done: false }
owned.next();   // { value: 2, done: false }
owned.next();   // { value: 3, done: false }
```

Async iterators work the same way:

```
async function* generate() { yield 1; yield 2; yield 3; }

const iter = generate();
const owned = Object.transfer(iter);

await iter.next();    // throws TypeError: AsyncIterator has been detached
await owned.next();   // { value: 1, done: false }
```

Design Principles

- Shape, not behavior – spec defines *when* the method is called, not *what* it does. Engine validates mechanics (callable, returns object) not semantics (source detached).
- Enables but does not enforce exclusive ownership – source is rendered unusable, but caller can still share the result. Not sharing is the caller's responsibility.
- Operational methods throw, cleanup methods no-op – `.next()`, `.read()`, etc. throw `TypeError` after transfer. `[Symbol.dispose]()` must silently no-op for safe `using` interaction.
- Opt-in, not universal – only objects implementing `[Symbol.transfer]()` are transferable. Without the symbol, `Object.transfer()` throws `TypeError`.
- Full MOP integration – `[[Transfer]]` internal method, `Object.transfer()`, `Reflect.transfer()`, Proxy `transfer` trap. Every fundamental operation has this triad.
- Frozen/sealed objects are non-transferable – transfer mutates observable behavior, contradicting `Object.freeze()`. Enforced by `[[Transfer]]`; direct `obj[Symbol.transfer]()` bypasses this.
- `Object.prototype` rejects `Symbol.transfer` – prevents `Object.prototype[Symbol.transfer] = fn` from making *every* object transferable. Follows the thenable curtailment model.

Interaction with `using`

Detached objects must not throw from dispose

```
async function example(source) {  
  await using iter = source[Symbol.asyncIterator]();  
  
  const owned = Object.transfer(iter);  
  // iter is now detached  
  
  doWork(owned);  
  
  // At block exit, iter[Symbol.asyncDispose]() is called.  
  // iter is detached, but this MUST NOT throw -- silent no-op.  
}
```

The correct pattern for both bindings:

```
await using iter = source[Symbol.asyncIterator]();  
await using owned = Object.transfer(iter);  
// now `owned` will be disposed at block exit
```

Use Case: Concurrent Safety

Make ownership explicit when passing async iterators between functions

```
async function processStream(source) {  
  const owned = Object.transfer(source);  
  // Any further use of `source` throws TypeError  
  for await (const chunk of owned) {  
    process(chunk);  
  }  
}
```

Use Case: `ReadableStream.from()` and Unprotected Iterators

The stream is locked – but the iterator underneath it isn't

```
const iter = generateData(); // async generator
const stream = ReadableStream.from(iter);
const reader = stream.getReader(); // stream is locked

// ...but nothing prevents consuming the iterator directly:
await iter.next(); // silently steals data from the stream
```

With `transfer`, `ReadableStream.from()` takes ownership of the iterator:

```
const iter = generateData();
const stream = ReadableStream.from(Object.transfer(iter));

await iter.next();           // throws TypeError: Iterator has been detached
await reader.read();         // 'chunk1' -- guaranteed
```

Applies to any API that wraps an iterator internally. Without `transfer`, the API must *trust* callers discard their reference. With `transfer`, it *enforces* it.

Use Case: Ownership in APIs

APIs that accept stateful objects can enforce the ownership contract

```
class DataProcessor {  
  #source;  
  
  constructor(source) {  
    this.#source = Object.transfer(source); // take ownership  
  }  
  
  process() {  
    for (const item of this.#source) {  
      // safe -- no other consumer can interfere  
    }  
  }  
}
```

Use Case: User-Defined Transferable Resources

Any class can participate in the protocol

```
class DatabaseCursor {
  #connection; #query; #detached = false;

  fetch() {
    if (this.#detached) throw new TypeError('Cursor has been detached');
    return this.#connection.fetchNext(this.#query);
  }

  [Symbol.transfer]() {
    if (this.#detached) throw new TypeError('Cannot transfer detached cursor');
    const cursor = new DatabaseCursor(this.#connection, this.#query);
    this.#connection = undefined;
    this.#query = undefined;
    this.#detached = true;
    return cursor;
  }
}

function processCursor(cursor) {
  const owned = Object.transfer(cursor); // enforce the contract
  while (true) { const row = owned.fetch(); if (!row) break; handle(row); }
}
```

Thank You

Transfer Protocol – Stage 0

`Symbol.transfer` | `Object.transfer()` | `Reflect.transfer()` | Proxy `transfer` trap

Seeking feedback on the core concept and API design